FramerMotion

Key Features of Framer Motion

- **Declarative Animations** : Motion uses a simple, declarative API, allowing developers to define animations directly in their JSX code, making it intuitive and easy to work with.
- **Smooth Transitions** : The library provides built-in support for smooth transitions between different states of a component (like opacity, position, scale, and rotation), ensuring a seamless user experience.
- **Gesture-based Animations** : Framer Motion allows you to trigger animations based on user gestures, such as hover, tap, drag, or while the user is scrolling. This creates more interactive and dynamic UI elements.
- **Page Transitions** : The library supports smooth page transitions, which can be especially useful in single-page applications (SPAs) to enhance the user experience when navigating between views or routes.
- **Customizable and Flexible** : Framer Motion offers extensive control over animations with properties like timing, easing functions, and variants, allowing you to create complex and customized animations that fit your needs.

Components of Framer Motion

- **Motion Components** : Motion components (e.g., <motion.div/>, <motion.circle/>) are the core of the Motion API, allowing you to animate any HTML or SVG element with additional props to handle gestures and animations easily.
- **Animate Presence** : This component is used for unmounting animations. It animates elements when they are removed from the React tree, providing smooth transitions during component removal.
- **Layout Group** : LayoutGroup groups motion components that should perform layout animations together, allowing for smooth transitions and animations when components change their layout within the parent container.
- **Lazy Motion** : LazyMotion helps reduce the bundle size by loading motion component features synchronously or asynchronously, optimizing performance by only loading the required features.

- **Reorder** : The Reorder component enables drag-to-reorder functionality, perfect for creating reorderable lists or items like to-do lists or tabs with simple drag-and-drop behaviour.

Motion Component, Its variants, AnimatePrescence [CODE]

```
import React, { useState } from "react";
import { motion, AnimatePresence } from "framer-motion";

const BasicsOfMotion = () => {
  const [isVisible, setIsVisible] = useState(true);
  return (
    <div
      style={{
        display: "grid",
        placeContent: "center",
        height: "100vh",
        gap: "0.8rem",
      }}
    >
      <motion.button
        onClick={() => setIsVisible(!isVisible)}
        className="example-button"
        layout
      >
        Show/Hide
      </motion.button>
      <AnimatePresence mode="popLayout">
        {isVisible && (
          <motion.div
            initial={{
              rotate: "0deg",
              scale: 0,
              y: 0,
            }}
            animate={{
              rotate: "180deg",
              scale: 1,
              y: [0, 150, -150, -150, 0],
            }}
            exit={{
              rotate: "0deg",
              scale: 0,
              y: 0,
            }}
            transition={{
              duration: 1,
              ease: "backInOut",
              times: [0, 0.25, 0.5, 0.85, 1],
            }}
```

```
            style={{
              width: 150,
              height: 150,
              background: "black",
            }}
          ></motion.div>
        )}
      </AnimatePresence>
    </div>
  );
};

export default BasicsOfMotion;
```

Gestures and MotionConfig [CODE]

```
import React from "react";
import { motion, MotionConfig } from "framer-motion";

const Gestures = () => {
  return (
    <div
      style={{
        display: "grid",
        placeContent: "center",
        height: "100vh",
        gap: "0.8rem",
      }}
    >
      <MotionConfig
        transition={{
          duration: 0.125,
          ease: "easeInOut",
        }}
      >
        <motion.button
          whileHover={{ scale: 1.05 }}
          whileTap={{ scale: 0.95, rotate: "2.5deg" }}
          className="example-button"
        >
          Click me!
        </motion.button>
        <motion.button
          whileHover={{ scale: 1.15 }}
          whileTap={{ scale: 0.85, rotate: "-2.5deg" }}
          style={{ backgroundColor: "red" }}
          className="example-button"
        >
          Click me!
        </motion.button>
      </MotionConfig>
    </div>
```

```
  );
};

export default Gestures;
```

Variants Example

```
const containerVariants = {
  hidden: { opacity: 0 },
  visible: {
    opacity: 1,
    transition: {
      staggerChildren: 0.2
    }
  }
};

const itemVariants = {
  hidden: { y: 20, opacity: 0 },
  visible: { y: 0, opacity: 1 }
};

<motion.ul
  variants={containerVariants}
  initial="hidden"
  animate="visible"
>
  <motion.li variants={itemVariants}>Item 1</motion.li>
  <motion.li variants={itemVariants}>Item 2</motion.li>
</motion.ul>
```

Variants with staggerChildren and delayChildren

`staggerChildren` is a **special property** used inside a **parent variant's** `transition` **object** . It tells Framer Motion to **delay the start of each child's animation by a specific amount of time** , creating a **staggered or cascading effect** .

- You apply `staggerChildren` inside the **parent's variant** , not on the children.
- In **Framer Motion** , it's very common (and recommended) to define the `transition` **inside a specific variant** , such as `"visible"`.

> Because transitions often depend on **what the target state is** (e.g., fade in, slide in), defining them inside the variant keeps the animation logic clean and localized

You can also use `delayChildren` in the parent to delay **when** the first child starts.

```
const containerVariants = {
  hidden: { opacity: 0 },
  visible: {
    opacity: 1,
    transition: {
      delayChildren: 0.5,
      staggerChildren: 0.2 // <-- Delay between each child
    }
  }
};

const itemVariants = {
  hidden: { opacity: 0, y: 20 },
  visible: { opacity: 1, y: 0 }
};

<motion.ul
  variants={containerVariants}
  initial="hidden"
  animate="visible"
>
  <motion.li variants={itemVariants}>Item 1</motion.li>
  <motion.li variants={itemVariants}>Item 2</motion.li>
  <motion.li variants={itemVariants}>Item 3</motion.li>
</motion.ul>
```

Below is the code where transition is applied normally which applies globally ie

```
<motion.div
  initial="hidden"
  animate="visible"
  variants={boxVariants}
  transition={{ duration: 0.5 }} // applies to all animated props
/>
```

View Based Animation

--useInView() HOOK

The `useInView()` hook from Framer Motion is a **React hook** that tells you **whether a DOM element is visible in the viewport**

Use `useInView()` when you want **programmatic control** over what happens when an element enters or exits the viewport.

Common use cases:

- Trigger animations manually (using `useAnimationControls`)
- Lazy loading content
- Updating UI state on scroll
- Tracking scroll-based progress

Example---

```
import React, { useEffect, useRef } from "react";
import { motion, useInView } from "framer-motion";

const ViewBasedAnimations = () => {
  const ref = useRef(null);
  const isInView = useInView(ref, { once: true }); // Has 2nd parameter for options

  useEffect(() => {
    console.log("Is in view -> ", isInView);
  }, [isInView]);

  return (
    <>
      <div style={{ height: "150vh" }} />
      <motion.div
        style={{ height: "100vh", background: "black" }}
        initial={{ opacity: 0 }}
        whileInView={{ opacity: 1 }}
        transition={{ duration: 1 }}
      />
      <div
        ref={ref}
        style={{
          height: "100vh",
          background: isInView ? "blue" : "red",
          transition: "1s background",
        }}
      />
    </>
  );
};

export default ViewBasedAnimations;
```

Option parameter of the hook(2nd parameter) can have 3 fields---

> once: true, // Only trigger once (default: false) margin: "0px", // Root margin (like CSS margin for viewport detection) amount: 0.5 // How much of the element must be visible (0 to 1 or "some", "all")

ANOTHER GOOD EXAMPLE---

```
import { motion, useAnimationControls, useInView } from "framer-motion";
import { useEffect, useRef } from "react";

const ScrollAnimateBox = () => {
  const ref = useRef(null);
  const isInView = useInView(ref, { once: true });
  const controls = useAnimationControls();

  useEffect(() => {
    if (isInView) {
      controls.start({ opacity: 1, y: 0, transition: { duration: 1 } });
    }
  }, [isInView, controls]);

  return (
    <motion.div
      ref={ref}
      animate={controls}
      initial={{ opacity: 0, y: 100 }}
      style={{ height: 100, backgroundColor: "salmon", marginBottom: 20 }}
    >
      I animate into view!
    </motion.div>
  );
};
```

Scroll Based Animations--

Example using useScroll(), useSpring(), useTransform()----

```
import { useScroll, motion, useSpring, useTransform } from "framer-motion";
import React from "react";

const ScrollAnimations = () => {
  const { scrollYProgress } = useScroll();

  const scaleX = useSpring(scrollYProgress);

  const background = useTransform(
```

```
    scrollYProgress,
    [0, 1],
    ["rgb(86, 1, 245)", "rgb(1, 245, 13)"]
  );

  return (
    <div>
      <motion.div
        style={{
          // scaleX: scrollYProgress,
          scaleX,
          transformOrigin: "left",
          // background: "blue",
          background,
          position: "sticky",
          top: 0,
          width: "100%",
          height: "20px",
        }}
      />

      <div
        style={{
          maxWidth: "700px",
          margin: "auto",
          padding: "1.2rem",
        }}
      >
        <p>
          Lorem Ipsum is simply dummy text of the printing and typesetting
          industry. Lorem Ipsum has been the industrys standard dummy text ever
          since the 1500s, when an unknown printer took a galley of type and
          scrambled it to make a type specimen book. It has survived not only
          five centuries, but also the leap into electronic typesetting,
          remaining essentially unchanged. It was popularised in the 1960s with
          the release of Letraset sheets containing Lorem Ipsum passages, and
          more recently with desktop publishing software like Aldus PageMaker
          including versions of Lorem Ipsum.
        </p>
        <p>
          Lorem Ipsum is simply dummy text of the printing and typesetting
          industry. Lorem Ipsum has been the industrys standard dummy text ever
          since the 1500s, when an unknown printer took a galley of type and
          scrambled it to make a type specimen book. It has survived not only
          five centuries, but also the leap into electronic typesetting,
          remaining essentially unchanged. It was popularised in the 1960s with
          the release of Letraset sheets containing Lorem Ipsum passages, and
          more recently with desktop publishing software like Aldus PageMaker
          including versions of Lorem Ipsum.
        </p>
        <p>
          Lorem Ipsum is simply dummy text of the printing and typesetting
          industry. Lorem Ipsum has been the industrys standard dummy text ever
          since the 1500s, when an unknown printer took a galley of type and
          scrambled it to make a type specimen book. It has survived not only
          five centuries, but also the leap into electronic typesetting,
          remaining essentially unchanged. It was popularised in the 1960s with
          the release of Letraset sheets containing Lorem Ipsum passages, and
```

```
      more recently with desktop publishing software like Aldus PageMaker
      including versions of Lorem Ipsum.
    </p>
    <p>
      Lorem Ipsum is simply dummy text of the printing and typesetting
      industry. Lorem Ipsum has been the industrys standard dummy text ever
      since the 1500s, when an unknown printer took a galley of type and
      scrambled it to make a type specimen book. It has survived not only
      five centuries, but also the leap into electronic typesetting,
      remaining essentially unchanged. It was popularised in the 1960s with
      the release of Letraset sheets containing Lorem Ipsum passages, and
      more recently with desktop publishing software like Aldus PageMaker
      including versions of Lorem Ipsum.
    </p>
    <p>
      Lorem Ipsum is simply dummy text of the printing and typesetting
      industry. Lorem Ipsum has been the industrys standard dummy text ever
      since the 1500s, when an unknown printer took a galley of type and
      scrambled it to make a type specimen book. It has survived not only
      five centuries, but also the leap into electronic typesetting,
      remaining essentially unchanged. It was popularised in the 1960s with
      the release of Letraset sheets containing Lorem Ipsum passages, and
      more recently with desktop publishing software like Aldus PageMaker
      including versions of Lorem Ipsum.
    </p>
    <p>
      Lorem Ipsum is simply dummy text of the printing and typesetting
      industry. Lorem Ipsum has been the industrys standard dummy text ever
      since the 1500s, when an unknown printer took a galley of type and
      scrambled it to make a type specimen book. It has survived not only
      five centuries, but also the leap into electronic typesetting,
      remaining essentially unchanged. It was popularised in the 1960s with
      the release of Letraset sheets containing Lorem Ipsum passages, and
      more recently with desktop publishing software like Aldus PageMaker
      including versions of Lorem Ipsum.
    </p>
    <p>
      Lorem Ipsum is simply dummy text of the printing and typesetting
      industry. Lorem Ipsum has been the industrys standard dummy text ever
      since the 1500s, when an unknown printer took a galley of type and
      scrambled it to make a type specimen book. It has survived not only
      five centuries, but also the leap into electronic typesetting,
      remaining essentially unchanged. It was popularised in the 1960s with
      the release of Letraset sheets containing Lorem Ipsum passages, and
      more recently with desktop publishing software like Aldus PageMaker
      including versions of Lorem Ipsum.
    </p>
    <p>
      Lorem Ipsum is simply dummy text of the printing and typesetting
      industry. Lorem Ipsum has been the industrys standard dummy text ever
      since the 1500s, when an unknown printer took a galley of type and
      scrambled it to make a type specimen book. It has survived not only
      five centuries, but also the leap into electronic typesetting,
      remaining essentially unchanged. It was popularised in the 1960s with
      the release of Letraset sheets containing Lorem Ipsum passages, and
      more recently with desktop publishing software like Aldus PageMaker
      including versions of Lorem Ipsum.
    </p>
```

```
        <p>
          Lorem Ipsum is simply dummy text of the printing and typesetting
          industry. Lorem Ipsum has been the industrys standard dummy text ever
          since the 1500s, when an unknown printer took a galley of type and
          scrambled it to make a type specimen book. It has survived not only
          five centuries, but also the leap into electronic typesetting,
          remaining essentially unchanged. It was popularised in the 1960s with
          the release of Letraset sheets containing Lorem Ipsum passages, and
          more recently with desktop publishing software like Aldus PageMaker
          including versions of Lorem Ipsum.
        </p>
        <p>
          Lorem Ipsum is simply dummy text of the printing and typesetting
          industry. Lorem Ipsum has been the industrys standard dummy text ever
          since the 1500s, when an unknown printer took a galley of type and
          scrambled it to make a type specimen book. It has survived not only
          five centuries, but also the leap into electronic typesetting,
          remaining essentially unchanged. It was popularised in the 1960s with
          the release of Letraset sheets containing Lorem Ipsum passages, and
          more recently with desktop publishing software like Aldus PageMaker
          including versions of Lorem Ipsum.
        </p>
        <p>
          Lorem Ipsum is simply dummy text of the printing and typesetting
          industry. Lorem Ipsum has been the industrys standard dummy text ever
          since the 1500s, when an unknown printer took a galley of type and
          scrambled it to make a type specimen book. It has survived not only
          five centuries, but also the leap into electronic typesetting,
          remaining essentially unchanged. It was popularised in the 1960s with
          the release of Letraset sheets containing Lorem Ipsum passages, and
          more recently with desktop publishing software like Aldus PageMaker
          including versions of Lorem Ipsum.
        </p>
        <p>
          Lorem Ipsum is simply dummy text of the printing and typesetting
          industry. Lorem Ipsum has been the industrys standard dummy text ever
          since the 1500s, when an unknown printer took a galley of type and
          scrambled it to make a type specimen book. It has survived not only
          five centuries, but also the leap into electronic typesetting,
          remaining essentially unchanged. It was popularised in the 1960s with
          the release of Letraset sheets containing Lorem Ipsum passages, and
          more recently with desktop publishing software like Aldus PageMaker
          including versions of Lorem Ipsum.
        </p>
      </div>
    </div>
  );
};

export default ScrollAnimations;
```

All props of Motion Components

These are the same props you'd pass to a regular `<div>`:

- `className`, `style`, `id`, `onClick`, `ref`, etc.
- All HTML attributes like `role`, `tabIndex`, `aria-*`, etc.

Animation Props---

Prop	Description
`animate`	Triggers animation to a target state or variant
`initial`	Sets initial style or variant before animation begins
`exit`	Defines exit animation (used with `AnimatePresence`)
`variants`	Defines named states for `initial`,`animate`, etc.
`transition`	Controls animation timing, delay, duration, easing, etc.
`whileHover`	Animation or variant to apply when hovered
`whileTap`	Animation or variant for when tapped/clicked
`whileDrag`	Animation or variant during drag
`whileFocus`	Animation or variant when focused
`whileInView`	Animation or variant triggered when element enters viewport
`layout`	Enables layout animations
`layoutId`	Used for shared layout transitions
`custom`	Passes custom data to variants
`onAnimationStart`,`onAnimationComplete`	Callbacks triggered at key points

Gesture Props---

Prop	Description
`drag`	Enables dragging (`true`,`"x"`,`"y"`)
`dragConstraints`	Limits how far an element can be dragged
`dragElastic`	Sets resistance to draggingz
`dragSnapToOrigin`	Snaps back after release
`dragMomentum`	Enables/disables momentum
`dragTransition`	Configures how it behaves during momentum/bounce. Uses inertia-specific properties.

Callback	When it Fires
`onHoverStart`	When pointer**enters**the element.
`onHoverEnd`	When pointer**leaves**the element.
`onTapStart`	When user**presses down**(mouse or touch).
`onTapCancel`	If the user**releases outside**the element.
`onTapEnd`	When user**releases** , regardless of cancel/tap result.
`onTap`	When press and release happen**inside**the element.
`onDragStart`,`onDrag`,`onDragEnd`	Drag lifecycle callbacks

✨ **Why use Framer Motion's versions?**

- `onHoverStart` is same as `onMouseEnter` , `onHoverEnd` same as `onMouseLeave`, `onTapStart` same as `onMouseDown`, `onTapEnd` same as `onMouseUp`
- But Framer Motion's versions are **gesture-aware** , meaning they work **consistently across devices** , including touch-enabled screens (where applicable).
- They also **integrate better** with `whileHover or whileTap`, allowing you to combine animations declaratively and imperatively.
- Also are designed to work perfectly with motion effects.

Scroll and view props----

Prop	Description
`whileInView`	Animation triggered when in viewport
`viewport`	Options for `whileInView`(e.g.,`{ once: true, amount: 0.5 }`)

Layout and Shared Transitions---

Prop	Description
`layout`	Enables auto-animated layout transitions
`layoutId`	Used for shared element transitions between screens/components
`layoutScroll`	Animates when scroll position/layout changes

MotionValue And Style Binding----

Prop	Description
`style`	Accepts `MotionValue`s and updates in REAL TIME
`transformTemplate`	Customize how `transform`CSS is generated

Motion Component- Layout Props And Shared Transitions

The `layout` prop in **Framer Motion** enables **automatic layout animations** — that is, when a component's **position or size changes** , Framer Motion animates it smoothly instead of snapping instantly.

-- Normally in React, when you update the layout (e.g., resize, reposition, add/remove elements), DOM changes happen **instantly** .

Framer Motion's `layout`:

- Detects **before and after** positions
- Automatically **animates the difference**
- Requires **no manual** `animate` **or** `variants`

When you use `layout`:

*--Framer Motion reads the **bounding box (position + size)** before and after the DOM change*

--Then uses a `transform` animation to interpolate the change

Example--

```
<motion.div layout>
  {isOpen && <motion.p layout>Expanded content here</motion.p>}
</motion.div>
```

When `isOpen` toggles, Framer Motion:

- Measures the size/position **before** the DOM updates
- Applies a transition **between the old and new layout**

Common Use Cases--

Use Case	What `layout`Does
Accordion toggle	Smooth height animation when expanding/collapsing
Reordering grid or list	Animates items to new positions
Dynamic UI resizing	Smooth transitions as dimensions or content change

-----HOW TO USE IT----

***-- On size-changing elements:**

```
<motion.div layout>
  {isExpanded && <p>More content</p>}
</motion.div>
```

-- On items that change order or position:

```
{items.map(item => (
  <motion.div key={item.id} layout>
    {item.text}
```

```
  </motion.div>
))}
```

Optional Props

- ***layoutTransition* **- To apply transition as the layout change
- ***layoutId* **- For **shared layout transitions** across routes or components. When two elements with the same `layoutId` mount/unmount (e.g., modal open/close), Framer Motion animates between them.

Example 1--

```
const [open, setOpen] = useState(false);

return (
  <motion.div layout>
    <button onClick={() => setOpen(!open)}>Toggle</button>
    {open && (
      <motion.div layout style={{ background: "#eee", padding: 20 }}>
        Expanding content!
      </motion.div>
    )}
  </motion.div>
);
```

Example 2--

```
import React, { useState } from "react";
import { motion, AnimatePresence } from "framer-motion";

const LayoutExample = () => {
  const [selectedId, setSelectedId] = useState(null);

  return (
    <div>
      {/* Card List */}
      <div style={{ display: "flex", gap: "1rem" }}>
        {[1, 2, 3].map(id => (
          <motion.div
            key={id}
            layoutId={`card-${id}`} // shared layout ID
            onClick={() => setSelectedId(id)}
            style={{
              width: 100,
              height: 120,
```

```
              background: "lightblue",
              borderRadius: 16,
              padding: 10,
              cursor: "pointer"
            }}
          >
            <motion.h4 layoutId={`title-${id}`}>Card {id}</motion.h4>
          </motion.div>
        ))}
      </div>

      {/* Modal View */}
      <AnimatePresence>
        {selectedId && (
          <motion.div
            layoutId={`card-${selectedId}`}
            style={{
              position: "fixed",
              top: 50,
              left: "25%",
              width: 300,
              background: "white",
              borderRadius: 20,
              padding: 20,
              boxShadow: "0 10px 30px rgba(0,0,0,0.2)",
              zIndex: 10
            }}
            layout
            layoutTransition={{ duration: 0.5, ease: "easeInOut" }}
            onClick={() => setSelectedId(null)}
          >
            <motion.h2 layoutId={`title-${selectedId}`}>Card {selectedId}
</motion.h2>
            <p>This is expanded modal content for card {selectedId}.</p>
          </motion.div>
        )}
      </AnimatePresence>
    </div>
  );
};

export default LayoutExample;
```

*The above is the shared transition between a **card view** and a **modal view** using **Framer Motion** .*

When you click on a card:

- It opens a modal.
- The image and title smoothly animate from the card to the modal using `layoutId`. Matches card and modal elements coz of shared transition
- The container size change is animated using `layout` and `layoutTransition`.

Therefore these happens --

> Cards animate into a modal with smooth motion + Title and container seamlessly morph using `layoutId`. + Modal expands/resizes smoothly using `layout`.

LayoutGroup

The `layoutGroup` prop in **Framer Motion** is used to group multiple `motion` components so that **layout animations are coordinated across them** . It's especially useful for animations involving **shared layout transitions** — like moving an element from one part of the DOM to another smoothly.

Use `layoutGroup` when:

- You want elements in different parts of the DOM to animate smoothly as if they are connected.
- You're using `layoutId` to animate between components during route/page transitions or component swaps.

--Without `LayoutGroup`, two elements with the same `layoutId` in different parts of the DOM won't animate smoothly — Framer Motion needs the group to coordinate those transitions.

EXAMPLE

Take these accordion items that each handle their own state--

```
function Item({ header, content }) {
  const [isOpen, setIsOpen] = useState(false)

  return (
    <motion.div
      layout
      onClick={() => setIsOpen(!isOpen)}
    >
      <motion.h2 layout>{header}</motion.h2>
      {isOpen ? content : null}
    </motion.div>
  )
}
```

If we arrange these next to each other in an `Accordion`, when their state updates, their siblings have no way of knowing:If we arrange these next to each other in an

`Accordion`, when their state updates, their siblings have no way of knowing:

```
function Accordion() {
  return (
    <>
      <ToggleContent />
      <ToggleContent />
    </>
  )
}
```

This can be fixed by grouping both components with `LayoutGroup`:

```
import { LayoutGroup } from "motion/react"

function Accordion() {
  return (
    <LayoutGroup>
      <ToggleContent />
      <ToggleContent />
    </LayoutGroup>
  )
}
```

EXAMPLE--

```
import { motion, AnimatePresence, LayoutGroup } from "framer-motion";

const MyComponent = () => {
  const [selectedId, setSelectedId] = React.useState(null);

  return (
    <LayoutGroup>
      <motion.div layoutId="box" onClick={() => setSelectedId("box")}>
        Click me
      </motion.div>

      <AnimatePresence>
        {selectedId && (
          <motion.div layoutId="box" onClick={() => setSelectedId(null)}>
            I'm the same box in a different place!
          </motion.div>
        )}
      </AnimatePresence>
    </LayoutGroup>
  );
};
```

🔁 What Happens Here?

- When you click the first box, it disappears.
- Another box (with the same `layoutId`) appears.
- Because they are inside a `LayoutGroup`, Framer Motion animates **between their layouts** .

Motion Component- Custom Prop

In **Framer Motion** , the `custom` prop is a powerful way to **pass dynamic data to your animation variants** , making your animations reusable and more flexible.

The `custom` prop allows you to **pass a value to your variant functions** . This is useful when:

- You want different animations for different components or conditions
- You want to make your variants dynamic instead of hardcoding everything

Instead of defining variants as **static objects** , you define them as **functions** that receive the `custom` value-----Example----

```
const boxVariants = {
  animate: (direction) => ({
    x: direction === "right" ? 200 : -200,
    opacity: 1,
    transition: { duration: 1 }
  })
};

// Then, pass the custom value to the component----

<motion.div
  custom="right"
  initial={{ opacity: 0 }}
  animate="animate"
  variants={boxVariants}
/>

//Framer Motion will call the variant function with the value "right".
```

FULL EXAMPLE----

```
import { motion } from "framer-motion";
```

```
const slideVariants = {
  animate: (direction) => ({
    x: direction === "right" ? 200 : -200,
    opacity: 1,
    transition: { duration: 0.8 }
  }),
  initial: {
    opacity: 0,
    x: 0
  }
};

const SlidingBox = ({ direction }) => (
  <motion.div
    custom={direction}
    initial="initial"
    animate="animate"
    variants={slideVariants}
    style={{
      width: 100,
      height: 100,
      backgroundColor: "skyblue",
      margin: 20
    }}
  />
);

// Now in jsx, Each box animates differently based on its custom prop ------

<SlidingBox direction="left" />
<SlidingBox direction="right" />
```

Motion Component- onAnimationStart() and onAnimationComplete()

In **Framer Motion** , the props `onAnimationStart` and `onAnimationComplete` are **event handlers** that allow you to execute custom logic **when an animation starts or finishes** on a Motion Component.

onAnimationStart

- Fires **once** when the animation **begins** .
- Can be used to trigger things like loading states, logs, or class changes.

onAnimationComplete

- Fires **once** when the animation **ends** .
- Great for triggering follow-up animations, callbacks, enabling buttons, etc.

Useful In:

- Chaining animations
- Tracking animation state
- Syncing UI logic with animation lifecycle

NOTE-- These are not per-property; they fire once for the whole animation block. If you're animating multiple properties (like `x`, `opacity`, `scale`), the event runs when all animations are done.

Example-----

```
<motion.div
  initial={{ opacity: 0 }}
  animate={{ opacity: 1 }}
  transition={{ duration: 1 }}
  onAnimationStart={() => console.log("Animation started")}
  onAnimationComplete={() => console.log("Animation completed")}
/>
```

Motion Component- transformTemplate

In **Framer Motion** , the `transformTemplate` prop is an **advanced customization tool** that lets you **control the order and structure of the final `transform` CSS string** applied to a motion component.

-- It's a function that lets you **override how Framer Motion builds the CSS `transform` property** , which is usually auto-generated from animated values like `x`, `scale`, `rotate`, etc.

```
<motion.div
  animate={{ scale: 1.2 }}
  transformTemplate={(transformString, transformObject) => {
    return `${transformString} perspective(500px)`
  }}
/>
```

> ^ Here 'transformString' is auto generated wherever needed by Framer motion depending on the way things are animated or layouts or by the way we program it. We are just telling them that whatever be the transformString let it be before perspective(500px). Beacuse order of transform matters in css..
>
> *transform: translateX(100px) rotate(45deg);* is different and so is this---> *transform: rotate(45deg) translateX(100px);*

We could even tell it to completely leave the transformString let it be replaced by ours. Example--

```
<motion.div
  animate={{ x: 100, rotate: 45 }}
  transformTemplate={(transformString, t) =>
    `rotate(${t.rotate}deg) translateX(${t.x}px)`
  }
/>
```

Why Use `transformTemplate`?

1. **Reorder transforms** (important for rendering performance or effect)
2. **Add custom transforms** not covered by Framer Motion
3. **Debug or tweak precision** in complex animations

EXAMPLE PROGRAM----

```
import React, { useState } from "react";
import { motion } from "framer-motion";

const FlipCard = () => {
  const [flipped, setFlipped] = useState(false);

  return (
    <div
      onClick={() => setFlipped(!flipped)}
      style={{
        perspective: 1000, // for 3D effect
        width: 200,
        height: 200,
        cursor: "pointer",
      }}
    >
      <motion.div
        animate={{ rotateY: flipped ? 180 : 0 }}
        transition={{ duration: 0.8 }}
        transformTemplate={(transformString, transform) => {
          return `perspective(1000px) ${transformString}`;
```

```
          }}
          style={{
            width: "100%",
            height: "100%",
            backgroundColor: flipped ? "#f39c12" : "#3498db",
            color: "#fff",
            fontSize: "1.5rem",
            display: "flex",
            alignItems: "center",
            justifyContent: "center",
            borderRadius: "12px",
            backfaceVisibility: "hidden",
          }}
        >
          {flipped ? "Back" : "Front"}
        </motion.div>
      </div>
    );
};

export default FlipCard;
```

What's happening here?

- `rotateY` is animated to flip the card on click.
- `transformTemplate` injects `perspective(1000px)` **before** the default transform values.
- This creates a **true 3D flipping effect** , which wouldn't be as effective without `perspective`.

AnimatePresence in detail

`AnimatePresence` is a component that enables **exit animations** for components **when they're removed from the React tree** .

-- By default, when a component unmounts in React, it disappears immediately — **you never get a chance to animate it out** .

`AnimatePresence` solves this by:

1. Keeping the component temporarily "alive" when it should exit.
2. Playing the animation defined in the `exit` prop.
3. Actually unmounting it **after** the animation finishes.

Must Know---

- `AnimatePresence` **must wrap the conditional component** , not its parent.
- Always use a unique `key` if rendering a list of animated items.

Optional Props on `AnimatePresence`-----

Prop	Description
`initial`	If `false`, skips `initial` animation on first mount
`mode`	Controls how multiple items exit —`"sync"`,`"wait"`, etc.

initial-- Set to `false` if you **only** want enter/exit animations when something **is added or removed** , not on the initial mount. DEFAULT - true

mode-- Controls **how** components are animated when **multiple components enter/exit simultaneously** . VALUES---

Value	Behavior
`"sync"` (default)	Enter and exit animations play at the**same time** .
`"wait"`	**Wait** for exit animation to complete **before** animating the new component in.
`"popLayout"`	When layout animations are involved, it handles transitions more intuitively between components.

> With `mode="wait"`, the second component won't start its animation until the first finishes **exiting** .

EXAMPLE---

```
<AnimatePresence mode="wait">
  {isToggled ? (
    <motion.div
      key="a"
      initial={{ opacity: 0 }}
      animate={{ opacity: 1 }}
      exit={{ opacity: 0 }}
    />
  ) : (
    <motion.div
      key="b"
      initial={{ opacity: 0 }}
      animate={{ opacity: 1 }}
```

```
      exit={{ opacity: 0 }}
    />
  )}
</AnimatePresence>
```

Example----

```
<AnimatePresence initial={false}>
  {show && (
    <motion.div
      initial={{ opacity: 0 }}
      animate={{ opacity: 1 }}
      exit={{ opacity: 0 }}
    />
  )}
</AnimatePresence>
```

➡️ **On first load:** the component appears **without fading in coz of**

➡️ **On toggle:** it **does fade in and out** as expected coz of initial in <motion.div>

MotionConfig in detail

MotionConfig is a **context provider** component that lets you define **global settings** for all motion components (`motion.div`, `motion.button`, etc.) inside it.

> `<MotionConfig>` component in **Framer Motion** can configure **more than just the `transition` property** . It acts as a **context provider** that passes default settings down to all nested motion components.

✅ What You Can Control with `MotionConfig`:

- Global `transition` settings (duration, easing, etc.)
- `reducedMotion` preferences
- `layoutTransition` settings
- `dragTransition`
- `transformPagePoint` (for advanced gesture/control scenarios)

📦 Common Props of `MotionConfig`

Prop	Description
`transition`	Default transition applied to all children motion components.
`reducedMotion`	`"always"`/`"never"`/`"user"`– for accessibility (prefers-reduced-motion).
`layoutTransition`	Default layout transition for `layout`animations.
`dragTransition`	Default transition for drag behavior.
`transformPagePoint`	Used in custom gesture handling.

When to Use `MotionConfig------`

- You want consistent animation behavior across multiple motion components.
- You're tired of writing `transition={{ ... }}` everywhere.
- You want to respect system-level reduced motion settings.

> NOTE---- You can override any of these settings locally inside an individual `motion` component if needed.

EXAMPLE---

```
<MotionConfig reducedMotion="user">
  <motion.div
    initial={{ scale: 0 }}
    animate={{ scale: 1 }}
    transition={{ duration: 1 }}
  />
</MotionConfig>
```

> If the user has **"prefers reduced motion"** enabled in their OS, this will auto-disable the animation.

```
import React from "react";
import { motion, MotionConfig } from "framer-motion";

const App = () => {
  return (
    <MotionConfig
      transition={{ duration: 1, ease: "easeInOut" }}
      layoutTransition={{ type: "spring", stiffness: 300 }}
      dragTransition={{ bounceStiffness: 100, bounceDamping: 8 }}
      transformPagePoint={(point) => ({
        x: point.x + 10, // shift all gesture coordinates by 10px
        y: point.y + 10,
```

```
        })}
      >
        <div style={{ padding: 40 }}>
          {/* Motion with transition from MotionConfig */}
          <motion.div
            initial={{ opacity: 0, scale: 0.5 }}
            animate={{ opacity: 1, scale: 1 }}
            style={{
              width: 100,
              height: 100,
              backgroundColor: "salmon",
              marginBottom: 30,
            }}
          />

          {/* Layout animation with layoutTransition from MotionConfig */}
          <motion.div
            layout
            style={{
              width: 200,
              height: 100,
              backgroundColor: "skyblue",
              marginBottom: 30,
            }}
          />

          {/* Draggable with dragTransition from MotionConfig */}
          <motion.div
            drag
            dragConstraints={{ left: 0, right: 300 }}
            style={{
              width: 100,
              height: 100,
              backgroundColor: "lightgreen",
              cursor: "grab",
            }}
          />
        </div>
      </MotionConfig>
    );
};

export default App;
```

Transition Properties

🧩 Common Transition Properties

These work with both `tween` and `spring` where applicable:

Property	Type	Description
`type`	`"tween"`,`"spring"`,`"inertia"`	Defines the type of animation
`delay`	`number`	Delay before animation starts
`duration`	`number`	How long the animation runs (used in `tween`)
`ease`	`string`or `[number, number, number, number]`	Controls speed curve for `tween`
`repeat`	`number`	Number of times to repeat
`repeatType`	`"loop"`,`"reverse"`,`"mirror"`	Direction/style of repeating
`repeatDelay`	`number`	Delay between repeats
`times`	`number[]`	Array of keyframe progress values (`[0, 0.5, 1]`)

🎛️ Spring-specific Properties (`type: "spring"` only)

Property	Type	Description
`stiffness`	`number`	Higher = faster bounce back
`damping`	`number`	Higher = less oscillation
`mass`	`number`	-Affects inertia/momentum. -A **higher mass** makes the animation feel heavier — moves slower and takes longer to settle. -Combined with `stiffness` and `damping`, it affects the realism of the motion. `transition={{ type: "spring", mass: 2 }}`
`bounce`	`number`	Value between 0 and 1 for bounciness.`1` = maximum bounce (lots of overshooting/oscillation). `transition={{ type: "spring", bounce: 0.5 }}`
`restSpeed`	`number`	Speed below which spring is considered "at rest"
`restDelta`	`number`	Distance threshold for stopping

Property	Type	Description
`velocity`	`number`	-Initial velocity of the spring - The**initial speed** at which the object starts moving (pixels per second). -Gives the animation a "kick" from the start. -Used mostly when animating between dynamic values (like drag or gesture-based animations). `transition={{ type: "spring", velocity: 100 }}`

🔧 `stiffness` (default: `100`)

- **Defines** : How strong or tight the spring is — higher = faster, snappier motion.
- **Range** :
- 🔹 `10–50`: Very loose spring — slow, soft movement
- 🔸 `100` (default): Balanced feel
- 🔺 `200–500`: Very stiff, fast snap
- 🔻 `>500`: Extremely stiff — practically instant movement

🔧 `damping` (default: `10`)

- **Defines** : How much resistance (friction) the spring has — higher = less bounce
- **Range** :
- 🔹 `1–5`: Very bouncy (low damping)
- 🔸 `10` (default): Light bounce
- 🔺 `20–40`: Heavily damped, almost no bounce
- 🔻 `>50`: Overdamped, slow to rest, sluggish

EXAMPLE--- This will create a smooth bounce-back effect that feels realistic and dynamic.

```
<motion.div
  animate={{ y: 0 }}
  initial={{ y: 300 }}
  transition={{
    type: "spring",
    mass: 1.5,
    bounce: 0.6,
    velocity: 50,
    damping: 10,
    stiffness: 100
  }}
/>
```

Inertia-specific Properties (`type: "inertia"` only)

Property	Type	Description
`velocity`	`number`	Initial speed (px/s) — how fast it starts moving.`0.8`.
`power`	`number`	How much momentum is generated from velocity. Default is `0.8`.
`timeConstant`	`number`	Controls deceleration rate
`bounceStiffness`	`number`	Spring back stiffness. Spring stiffness when bouncing off a boundary.
`bounceDamping`	`number`	Damping when bouncing. How much resistance is applied when bouncing.
`min`/`max`	`number`	Boundaries for animation
`modifyTarget`	`fn`	Function to alter animation endpoint. How much resistance is applied when bouncing.

EXAMPLE--- (Manual inertia animation)

```
<motion.div
  animate={{ x: 500 }}
  transition={{
    type: "inertia",
    velocity: 200,
    min: 0,
    max: 500,
    bounceStiffness: 200,
    bounceDamping: 15
  }}
/>
```

EXAMPLE-- (Free glide with boundaries)--- using `dragTransition`

```
<motion.div
  animate={{ x: 0 }}
  drag="x"
  dragConstraints={{ left: -200, right: 200 }}
  dragTransition={{
    bounceStiffness: 300,
    bounceDamping: 20
  }}
```

```
/>
```

EXAMPLE-- Snap to grid using `modifyTarget`

```
<motion.div
  animate={{ x: 0 }}
  transition={{
    type: "inertia",
    velocity: 120,
    modifyTarget: (target) => Math.round(target / 100) * 100
  }}
/>
```

type property

The `type` property in a Framer Motion `transition` defines **how** the animation progresses — like whether it behaves like a spring, tween, or inertia animation.

```
transition={{ type: "spring" | "tween" | "inertia", ...otherOptions }}
```

1. ✅ `type: "tween"` (default for most properties)

- Uses **easing functions** (like `easeInOut`, `linear`)
- Best for **opacity** , **colors** , or precise timing

```
<motion.div
  animate={{ opacity: 1 }}
  transition={{ type: "tween", duration: 0.5, ease: "easeOut" }}
/>
```

2. ✅ `type: "spring"`

- Physics-based spring animation
- Good for **scale** , **position** , or bouncy UI effects
- > Ignores `duration`, instead uses `stiffness`, `damping`, etc.

```
<motion.div
  animate={{ x: 100 }}
  transition={{ type: "spring", stiffness: 200, damping: 20 }}
/>
```

- `stiffness`: how fast it tries to reach the target
- `damping`: how much it resists bouncing
- `mass`: affects momentum

3. ✅ `type: "inertia"`

- Animates with **momentum** (used mostly with gestures like `drag`)
- Best for **scroll** , **pan** , or **swipe-style** interactions

```
<motion.div
  drag="x"
  dragConstraints={{ left: 0, right: 300 }}
  dragTransition={{
    type: "inertia",
    bounceStiffness: 200,
    bounceDamping: 10
  }}
/>
```

> Used primarily inside `dragTransition`, not `transition`

ease Property

The `ease` property in a `transition` defines **how the animation progresses over time** — whether it starts slowly, ends slowly, or moves linearly.

-- It's only used with `type: "tween"` transitions. It's ignored in `spring` or `inertia`.

✅ Common Predefined Easing Options

Ease Type	Behavior
`"linear"`	Constant speed (no acceleration)
`"easeIn"`	Starts slow, ends fast

Ease Type	Behavior
`"easeOut"`	Starts fast, ends slow
`"easeInOut"`	Starts and ends slow, fast in the middle
`"circInOut"`	Circular easing (very smooth start and end)
`"backInOut"`	Slight overshoot in and out

You can define custom easing using cubic bezier values:

```
ease: [0.42, 0, 0.58, 1] // same as easeInOut
```

> **Can also be set individually per animated property:**

```
transition={{
  x: { duration: 1, ease: "easeOut" },
  opacity: { duration: 0.5, ease: "easeIn" }
}}
```

repeatType property

The `repeatType` controls **how** the animation repeats. You can use it along with the `repeat` prop to define repeat behavior

Options:

Value	Behavior
`"loop"`	Repeats from start to end over and over again (like a loop)
`"reverse"`	Repeats in reverse direction (end to start, then start to end, etc.)
`"mirror"`	Mirrors the animation like `reverse`, but preserves easing and direction

repeatDelay Property

The `repeatDelay` adds a **pause** between repeats — whether you're using `loop`, `reverse`, or `mirror`.

```
<motion.div
  animate={{ rotate: [0, 360] }}
  transition={{
    duration: 2,
    repeat: Infinity,
    repeatType: "loop",
    repeatDelay: 1  // 1 second delay between loops
  }}
/>
```

Gesture Event Callbacks

Framer Motion also gives **gesture event callbacks** :

- `onHoverStart`, `onHoverEnd`
- `onTapStart`, `onTapCancel`, `onTap`
- `onDragStart`, `onDragEnd`, `onDrag`

```
<motion.div
  onTap={() => console.log("Tapped!")}
  whileTap={{ scale: 0.95 }}
/>
```

All drag-related properties, transition options, and callbacks

Enables drag behavior---- `<motion.div drag />`

- `drag` → enables dragging on both axes.
- `drag="x"` or `drag="y"` → restricts drag to a single axis.

⚙️ Drag-related Props

Prop	Type	Description
`dragConstraints`	`object` or `ref`	Limits dragging within a box (`{ left, right, top, bottom }`) or a DOM element (`ref`).
`dragDirectionLock`	`boolean`	Locks drag direction after a short movement (x or y).
`dragElastic`	`boolean` or `number`	Controls "stretchiness" beyond constraints.`0`= no stretch;`1`= full elastic.
`dragMomentum`	`boolean`	Whether inertia is applied after drag ends. Default:`true`.
`dragTransition`	`object`	Configures how it behaves during momentum/bounce. Uses inertia-specific properties.
`dragSnapToOrigin`	`boolean`	If true, snaps back to the original position after dragging.

EXAMPLE----

```
<motion.div
  drag
  dragConstraints={{ left: -100, right: 100, top: -50, bottom: 50 }}
  dragElastic={0.2}
  dragMomentum={true}
  style={{ width: 100, height: 100, background: "skyblue" }}
/>
```

`dragTransition` (inertia / bounce)

--A sub-property used to customize how the element behaves **after drag ends** .

Common `dragTransition` options:

Property	Description
`bounceStiffness`	Spring stiffness when bouncing at edge
`bounceDamping`	Damping (resistance) applied when bouncing

Property	Description
`power`	Momentum power (higher = longer glide)
`timeConstant`	Time constant for exponential decay
`modifyTarget`	Function to change final resting point (e.g., snap)

```
<motion.div
  drag
  dragTransition={{
    bounceStiffness: 400,
    bounceDamping: 20
  }}
/>
```

 Drag Event Callbacks

You can use these to track drag lifecycle events---

Callback	Triggered When...
`onDragStart`	Drag begins
`onDrag`	While dragging
`onDragEnd`	Drag is released
`onDirectionLock`	When direction gets locked (x or y)

```
<motion.div
  drag
  onDragStart={() => console.log("Start dragging")}
  onDrag={(event, info) => console.log("Dragging", info.point)}
  onDragEnd={() => console.log("Drag ended")}
  onDirectionLock={(axis) => console.log("Locked to", axis)}
/>
```

 Extra Notes

- Combine drag with `animate` or `variants` for complex interactions.

- Use `useAnimationControls()` with `onDragEnd` to trigger custom effects.
- You can use `ref` for `dragConstraints` to dynamically define the container.

```
<motion.div
  drag
  dragConstraints={{ left: -100, right: 100 }}
  dragElastic={0.5}
  dragTransition={{ bounceStiffness: 300, bounceDamping: 20 }}
  onDragEnd={(e, info) => {
    console.log("Velocity", info.velocity);
    console.log("Offset", info.offset);
  }}
  style={{
    width: 100,
    height: 100,
    borderRadius: 10,
    background: "salmon"
  }}
/>
```

Animate props of Motion Component--Note

- `animate` overrides `whileHover`, `whileTap`, etc. if they conflict.
- It should always be used with `initial` for entrance animations.

Framer Motion **automatically builds the `transform` property** for you from values like:

- `x`, `y` → `translateX`, `translateY`
- `scale`, `scaleX`, `scaleY`
- `rotate`, `rotateX`, `rotateY`
- `skewX`, `skewY`
- `perspective`, etc.

This keeps the syntax clean and declarative, and also allows **fine-grained animation control** over individual transform properties.

Reorder

The `Reorder` components can be used to create drag-to-reorder lists, like reorderable tabs or todo items.

--Reorder refers to a **specialized way of animating the reordering of a list** when the order of items changes — especially useful for things like drag-to-reorder lists or sortable grids.

`reorder` is a feature from Framer Motion's `framer-motion` package that helps:

- Animate **position changes** when items move in the DOM.
- Handle **drag-and-drop reordering** smoothly.
- Reduce the boilerplate for managing animated lists.

It works in conjunction with:

- `<Reorder.Group>`: the container for reorderable items.
- `<Reorder.Item>`: each item in the list.

EXAMPLE--

```
import React, { useState } from "react";
import { Reorder } from "framer-motion";

export default function ReorderExample() {
  const [items, setItems] = useState(["🍎", "🍌", "🍇", "🍑"]);

  return (
    <Reorder.Group axis="y" values={items} onReorder={setItems}>
      {items.map((item) => (
        <Reorder.Item key={item} value={item}>
          {item}
        </Reorder.Item>
      ))}
    </Reorder.Group>
  );
}
```

Prop / Component	Purpose
`Reorder.Group`	A wrapper that enables drag-to-reorder for its children
`axis="y"`	Constrains drag to vertical (use `x`for horizontal)
`values`	The array of items representing order
`onReorder={setItems}`	Callback that updates the list order after reordering

Prop / Component	Purpose
`Reorder.Item`	The draggable item component with `value`representing its identity

🛠 How It Works

1. You provide a list of items with `values`.
2. Framer Motion tracks the order of `Reorder.Item`s.
3. When you drag an item, it:
    - Automatically updates the layout.
    - Animates the position of items as they move.
4. `onReorder` is called with the new order, which you apply using `setItems`.

🧩 Use Cases

- Drag-to-reorder lists (e.g. task managers, kanban boards)
- Animated sort transitions
- Playlists or any ordered collections with user interaction

++ By default, this is rendered as a `<ul>`, but this can be changed with the `as` prop.

`<Reorder.Group as="ol">`

Scrollable List

If `Reorder.Item` components are within a scrollable container, that container needs a `layoutScroll` prop so Framer Motion can correctly measure its scroll offset.

```
<Reorder.Group
  axis="y"
  onReorder={setItems}
  layoutScroll
  style={{ overflowY: "scroll" }}
>
  {items.map((item) => (
    <Item key={item} item={item} />
  ))}
</Reorder.Group>
```

reducedMotion

In **Framer Motion** , `reducedMotion` is a feature that **respects the user's system preferences** for reduced motion (like in accessibility settings). It's useful for creating inclusive animations that **automatically disable or simplify motion** for users who prefer less animation.

🔧 Two Ways to Use `reducedMotion`:

1. **Using the `useReducedMotion()` Hook**

EXAMPLE---

```
import { useReducedMotion, motion } from "framer-motion";

const MyComponent = () => {
  const shouldReduceMotion = useReducedMotion();

  return (
    <motion.div
      animate={{
        x: shouldReduceMotion ? 0 : 100,
        opacity: shouldReduceMotion ? 1 : [0, 1],
      }}
      transition={{ duration: shouldReduceMotion ? 0 : 1 }}
    >
      I respect your motion preferences!
    </motion.div>
  );
};
```

2. **Using `<MotionConfig reducedMotion="user" />`**

EXAMPLE--

```
import { MotionConfig, motion } from "framer-motion";

const App = () => {
  return (
    <MotionConfig reducedMotion="user">
      <motion.div
        animate={{ x: 200 }}
        transition={{ duration: 1 }}
      >
        Will skip animation if reduced motion is on.
```

```
        </motion.div>
      </MotionConfig>
    );
  };
```

You can also pass:

- `"always"` → always reduce motion (even if user didn't request it)
- `"never"` → always allow full motion (ignores user settings)
- `"user"` → default; respect system preference

🔍 Why It Matters

Some users experience motion sickness or discomfort from excessive or complex animations. `reducedMotion` helps **gracefully degrade animations** , improving **accessibility** without you needing to write separate animation logic.

OTHER CONCEPTS

1. LazyMotion

`LazyMotion` is a **performance optimization** tool that lets you **lazy-load only the parts of Framer Motion you need** , instead of shipping the full animation runtime (which includes a lot of extra features like SVG path animation, layout animations, etc.).

2. transformPagePoint() prop of MotionConfig

`transformPagePoint` is an **optional function** you can provide to customize how pointer coordinates (like mouse/touch positions) are transformed before Framer Motion uses them—for example, when dragging.

Example---

```jsx
<MotionConfig
  transformPagePoint={(point) => {
    return {
      x: point.x - window.scrollX,
      y: point.y - window.scrollY
    };
  }}
>
  <motion.div drag />
</MotionConfig>
```

HOOKS

1.useAnimationControls()

The `useAnimationControls` hook gives you **full programmatic control** over animations. Instead of relying only on `initial` / `animate` props, this hook lets you **start, stop, or change animations manually** from within your component logic.

Use `useAnimationControls` when:

- You want to trigger animations based on user actions (e.g., onClick, onScroll).
- You need **fine-grained control** over what animates and when.
- You're orchestrating complex animations across components.

For **simple interaction-based animations** like `whileHover`, `whileTap`, etc., you **don't need** `useAnimationControls` . You can just define a `variant` and let Framer Motion handle it declaratively.

Use `useAnimationControls` when:

1. You need to trigger animations imperatively - e.g., on a button click, scroll event, data load, or timeout.
2. The animation depends on external logic or state- Example: animation only after fetching data from an API
3. You're coordinating multiple components programmatically- An entire page or multiple elements animating sequentially ie Animation orchestration

```
import { motion, useAnimationControls } from "framer-motion";

function MyComponent() {
  const controls = useAnimationControls();

  const handleClick = () => {
    controls.start({
      x: 100,
      opacity: 1,
      transition: { duration: 0.5 }
    });
  };

  return (
    <>
      <motion.div
        animate={controls} // Hook is connected here
        initial={{ x: 0, opacity: 0 }}
        style={{ width: 100, height: 100, background: 'blue' }}
      />
      <button onClick={handleClick}>Animate</button>
    </>
  );
}
```

- If you already hava a variant you could use that inside controls.start()--- controls.start("visible")
- To stop current animation-- controls.stop();
- Immediately set values (without animation)-- controls.set({ x: 0, opacity: 1 });

Another Example USING .start(), .set(), .stop() and pssing variants inside.start()

```
import React from "react";
import { motion, useAnimationControls } from "framer-motion";

const boxVariants = {
  hidden: { x: 0, opacity: 0 },
  visible: {
    x: 300,
    opacity: 1,
    transition: { duration: 2 }
  }
};

const AnimationControlExample = () => {
  const controls = useAnimationControls();

  const startAnimation = () => {
    controls.start("visible"); // Triggering the variant
  };
```

```
  const stopAnimation = () => {
    controls.stop();
  };

  const setPosition = () => {
    controls.set({
      x: 50,
      opacity: 0.5
    });
  };

  return (
    <div style={{ padding: 20 }}>
      <motion.div
        variants={boxVariants}         // Apply variants
        initial="hidden"               // Start from hidden
        animate={controls}             // Controlled by useAnimationControls
        style={{
          width: 100,
          height: 100,
          backgroundColor: "skyblue",
          marginBottom: 20
        }}
      />

      <button onClick={startAnimation} style={{ marginRight: 10 }}>
        Start Animation (Variant)
      </button>
      <button onClick={stopAnimation} style={{ marginRight: 10 }}>
        Stop Animation
      </button>
      <button onClick={setPosition}>Set Position Instantly</button>
    </div>
  );
};

export default AnimationControlExample;
```

* 2.useScroll()

The `useScroll()` hook in **Framer Motion** lets you **track scroll progress** of either:

- the **whole page** , or
- a specific **element** (when used with a `ref`).

This is especially useful for building scroll-based animations, progress indicators, or triggering transitions at different scroll positions.

The Values returned by the hook

> scrollX, // motionValue <number> for horizontal scroll (pixels) scrollY, // motionValue <number> for vertical scroll (pixels) scrollXProgress, // motionValue <number> from 0 to 1 scrollYProgress // motionValue <number> from 0 to 1

Optional paramter of useScroll---

```
const { scrollYProgress } = useScroll({
  container: ref,        // optional scroll container
  target: targetRef,     // optional tracking target within container
  offset: ["start end", "end start"]  // customize trigger points. Offset lets you
define when the progress should start/end
});
```

Best Use Cases

- Scroll-based progress bars
- Animating elements as they scroll into view
- Pinning sections
- Parallax effects

Tracking Scroll Within an Element using optional "container" of the hook---

```
import { useRef } from "react";
import { useScroll } from "framer-motion";

const ScrollInElement = () => {
  const containerRef = useRef(null);
  const { scrollYProgress } = useScroll({ container: containerRef });

  return (
    <div ref={containerRef} style={{ height: 300, overflowY: "scroll" }}>
      <motion.div style={{ scaleY: scrollYProgress, height: "100%" }}>
        Scrolling inside a div
      </motion.div>
    </div>
  );
};
```

--`target` and `offset` in `useScroll()`

`-target`

- A **ref** to an element whose scroll position (relative to its container or viewport) you're tracking.
- Used when you want to track **how far** an element has entered/exited the viewport or container.

-offset

- Defines **when** scroll progress should **start and end** .
- Accepts two values (start and end points), e.g.:
- It's like saying:
  - **"Start when the top of target hits the bottom of viewport/container"** (`start end`)
  - **"End when the bottom of target hits the top of viewport/container"** (`end start`)

Example---

```
import { useRef } from "react";
import { useScroll, motion, useTransform } from "framer-motion";

const ScrollSection = () => {
  const targetRef = useRef(null);
  const { scrollYProgress } = useScroll({
    target: targetRef,
    offset: ["start end", "end start"]
  });

  const scale = useTransform(scrollYProgress, [0, 1], [0.5, 1]);

  return (
    <div style={{ height: "200vh", paddingTop: "100vh" }}>
      <motion.div
        ref={targetRef}
        style={{
          background: "lightgreen",
          height: 300,
          scale
        }}
      >
        I scale while being scrolled into view
      </motion.div>
    </div>
  );
};
```

* 3. useInview()

See View Based Animation

* 4. useTransform()

The `useTransform()` hook in **Framer Motion** is a powerful utility that lets you map one `MotionValue` to another — transforming a value from one range into another, often used in scroll or animation contexts.

-- One of the most powerful tools for creating **smooth, scroll-based or reactive animations** .

```
const output = useTransform(input, inputRange, outputRange);
```

Argument	Description
`input`	A `MotionValue`(like `scrollY`,`scrollYProgress`,`x`, etc.)
`inputRange`	An array defining the range of the input (e.g.`[0, 1]`, or `[0, 100]`)
`outputRange`	An array defining how the output should map to the input (e.g.`[0, 100]`)

- The value returned by `useTransform()` is a **live MotionValue** , so it updates automatically as the input changes.
- You can use it directly in styles or pass it to other Framer components.

Use Case	What to Transform
Scroll progress bar	`scaleX`
Parallax scrolling	`y`,`x`,`opacity`,`scale`
Changing background color on scroll	`backgroundColor`
Rotate or move elements on drag	`x`,`rotate`,`scale`
Dynamic blur or box-shadow	`filter`,`boxShadow`

EXAMPLE---

```jsx
import { motion, useScroll, useTransform } from "framer-motion";

const ScrollFade = () => {
  const { scrollYProgress } = useScroll(); // 0 to 1

  const opacity = useTransform(scrollYProgress, [0, 1], [0.2, 1]);
  const scale = useTransform(scrollYProgress, [0, 1], [0.8, 1]);

  return (
    <motion.div style={{ opacity, scale, height: "100vh", background: "teal" }}>
      Scroll to Animate Me
    </motion.div>
  );
};
```

EXAMPLE (Link a drag value to a rotate value)---

```jsx
const x = useMotionValue(0);
const rotate = useTransform(x, [-200, 200], [-45, 45]);

<motion.div drag="x" style={{ x, rotate }} />
```

5. useSpring()

The `useSpring` hook in **Framer Motion** lets you create a **smooth, physics-based animated value** that follows another `MotionValue` — like a spring attached to it.

✅ What it does

It creates a new `MotionValue` that:

- Tracks a **source MotionValue**
- Animates smoothly toward it using **spring physics**
- Automatically responds to changes in the source

-- If you want to **smooth out a jerky or abrupt animation** , `useSpring` gives it a **natural, bouncy feel** — like how real objects move due to inertia and resistance.

EXAMPLE---

```jsx
import { motion, useMotionValue, useSpring } from "framer-motion";

export default function SpringExample() {
  const x = useMotionValue(0);
  const springX = useSpring(x, {
    stiffness: 100,
    damping: 10,
  });

  return (
    <motion.div
      drag="x"
      style={{
        x: springX, // smoother version of x
        width: 100,
        height: 100,
        background: "hotpink",
        borderRadius: 20,
      }}
    />
  );
}
```

> **THE OPTION OBJECT CAN HAVE SAME PROPERTIES AS THAT OF when type: 'spring' is used in any motion component**

🎯 **Real Use Cases**

- Smooth scrolling indicators
- Follow-the-cursor effects with damping
- Sticky UI elements with realistic delay

🔄 **You can also create a spring manually:**

```jsx
const smoothOpacity = useSpring(initialValue, springOptions);
```

And then update it manually: `smoothOpacity.set(1); // triggers a spring animation to 1`

6. useMotionValue()

The `useMotionValue` hook in **Framer Motion** is a powerful tool that lets you create **reactive values** tied to animation. These values are not part of React's normal state system, which allows for **high-performance updates** —ideal for fluid motion.

🔧 What is `useMotionValue`?

`useMotionValue` creates a special value that can:

- Be animated directly (e.g. via `animate`, `drag`, `set`, or `spring`)
- Be read and updated efficiently (***without triggering React re-renders***)
- Be tracked in real-time (e.g. to link scroll or gestures to style)

```
import React from "react";
import { motion, useMotionValue } from "framer-motion";

export default function MotionValueExample() {
  const x = useMotionValue(0);

  return (
    <motion.div
      drag="x"
      style={{ x, width: 100, height: 100, background: "tomato" }}
    />
  );
}
```

Explanation:

- `x` is a motion value.
- You pass it into `style` as `x: x`.
- When the user drags the element, Framer Motion updates `x` in real time **without re-rendering** .

SYNTAX ------ `const motionValue = useMotionValue(initialValue);`

🔁 Common Uses

Use Case	Description
`drag`values	Capture real-time drag position (e.g.,`x`,`y`)
Custom animation controls	Animate values manually using `motionValue.set()`or `.animate()`

Use Case	Description
Derived animations	Use `useTransform()` to reactively map values (e.g., scroll position to scale or color)
Sync with DOM styles	Apply live styles like blur, opacity, etc., without rerenders

EXAMPLE----- Changing Background with Drag----

```
import React from "react";
import { motion, useMotionValue, useTransform } from "framer-motion";

export default function DragColorChange() {
  const x = useMotionValue(0);
  const background = useTransform(x, [-200, 0, 200], ["#f00", "#fff", "#0f0"]);

  return (
    <motion.div
      drag="x"
      style={{
        x,
        background,
        width: 100,
        height: 100,
        borderRadius: 20
      }}
    />
  );
}
```

NOTE---

- It avoids React's rendering cycle for smooth animation.
- Can be combined with `useTransform`, `scroll`, `drag`, or manual updates.

7. useMotionTemplate

The `useMotionTemplate` hook in **Framer Motion** is used to **create dynamic CSS strings** from `MotionValue`s — especially helpful for styles that can't be passed as plain objects like `transform`, `filter`, `background`, etc.

--Same like useMotionValues(), but that cannot generate a string dynamically without triggering re-renders which then can be passed in properties lke rotate, filter,

background, etc where it expects string value

SYNTAX---- `const template = useMotionTemplate `rotate(${rotate}deg) scale(${scale})`;`

In the above,

- `${rotate}` and `${scale}` are `MotionValue`s.
- `template` is a derived `MotionValue<string>` that updates automatically.

EXAMPLE-- Dynamic `filter` with drag

```
import React from "react";
import { motion, useMotionValue, useMotionTemplate } from "framer-motion";

export default function BlurOnDrag() {
  const x = useMotionValue(0);
  const blur = useMotionTemplate`blur(${x}px)`;

  return (
    <motion.div
      drag="x"
      style={{
        x,
        filter: blur,
        width: 150,
        height: 150,
        backgroundColor: "skyblue",
        borderRadius: 20
      }}
    />
  );
}
```

In the above,

- As you drag, `x` updates.
- The `blur` value becomes `blur(0px)`, `blur(20px)`, etc. — depending on drag.
- The element **gets blurrier as you drag it** .

🧩 Good Use Cases

- `filter`: e.g. `blur`, `brightness`, `hue-rotate`
- `transform`: advanced custom transforms
- `background`: e.g., `linear-gradient(${x}px, red, blue)`

🚀 **Benefits**

- Reactive
- Efficient (no re-render)
- Works with `MotionValue` seamlessly
- Great for performance-sensitive UI

8. useMotionValueEvent()

The `useMotionValueEvent` hook in **Framer Motion** lets you **subscribe to changes in a `MotionValue`** (like `x`, `opacity`, `scrollY`) and **run side effects** when it changes — **without causing re-renders** .

🧠 Why use it?

Normally, if you use `useEffect` with `.on("change")`, you have to manage subscriptions manually.

`useMotionValueEvent` simplifies this by:

- Automatically subscribing and cleaning up
- Letting you run a **callback function** on each value update
- Keeping your component performant (no re-renders unless **you** trigger one)

SYNTAX---

```
useMotionValueEvent(motionValue, "change", (latest) => {
  // Do something with the updated value
});
```

EXAMPLE--- Logging drag position

```
import { motion, useMotionValue, useMotionValueEvent } from "framer-motion";

export default function MotionValueEventExample() {
  const x = useMotionValue(0);

  useMotionValueEvent(x, "change", (latest) => {
    console.log("x changed to:", latest);
```

```
  });

  return (
    <motion.div
      drag="x"
      style={{
        x,
        width: 100,
        height: 100,
        background: "salmon",
        borderRadius: 20,
      }}
    />
  );
}
```

🟢 As you drag the box, it logs the current `x` value in real-time — without re-rendering the component.

> **You can do the same with useEffect(), but wont be clean and has certain drawbacks.**

```
useEffect(() => {
  const unsubscribe = x.on("change", (latest) => {
    console.log("x changed to:", latest);
  });
  return unsubscribe;
}, [x]);
```

❗ **So why use `useMotionValueEvent()`?**

Because it's **purpose-built** for MotionValue subscriptions and gives you:

Feature	`useEffect()` **with** `.on("change")`	`useMotionValueEvent()` ✅
Cleaner, declarative syntax	❌ More manual setup	✅ Yes
Auto cleanup	✅ (but you write it)	✅ (handled internally)
Supports `animationStart` & `animationComplete`	❌ No	✅ Yes

Feature	`useEffect()` with `.on("change")`	`useMotionValueEvent()` ✅
No need to track `motionValue` deps	✅	✅
Tailored for Framer Motion use case	❌ General-purpose	✅ Purpose-built

HENCE---

- For **simple one-off cases** , `useEffect` + `.on("change")` is fine.
- For **cleaner, readable, and Framer-native code** , especially when using more than one event type or multiple `MotionValue`s — use `useMotionValueEvent`.

🎯 Real-World Use Cases

- Sync motion values with **external libraries** (e.g., `gsap`, audio, custom DOM animations)
- Show/hide UI based on scroll position (e.g., navbar hiding on scroll)
- Trigger analytics/events at thresholds (e.g., `if (x > 200) fireEvent()`)

🔁 Supported Events

Event	Description
`"change"`	Fires on every value change
`"animationStart"`	Fires when an animation begins
`"animationComplete"`	Fires when animation ends

9. useAnimate()

The `useAnimate` hook in **Framer Motion v7+** is a **powerful and low-level** way to trigger animations directly on DOM elements using **imperative code** — like calling `element.animate(...)`, but with Framer Motion's powerful animation engine.

✅ What is `useAnimate`?

It's a React hook that gives you:

1. A **ref** to attach to any DOM element.

2. A `animate()` **function** to trigger animations on that element (or its children).
3. A `scope` **object** to target child elements easily by selector.

🧠 When to use it?

Use `useAnimate` when:

- You need **fine-grained control** over when/how things animate (e.g., after a button click).
- You want to **animate multiple elements together** imperatively.
- You don't want to tie animation to React state/props.

SYNTAX------ `const [scope, animate] = useAnimate();`

- `scope` → attach to the element (via `ref`)
- `animate()` → call to trigger animation

EXAMPLE---

```
import { useAnimate } from "framer-motion";
import { useEffect } from "react";

export default function Example() {
  const [scope, animate] = useAnimate();

  useEffect(() => {
    animate(scope.current, { opacity: 1, x: 100 }, { duration: 1 });
  }, []);

  return (
    <div ref={scope} style={{ opacity: 0, background: "lightblue", width: 100,
height: 100 }} />
  );
}
```

EXAMPLE-- With button and Staggered Children

```
import { useAnimate } from "framer-motion";

export default function StaggerExample() {
  const [scope, animate] = useAnimate();

  const handleClick = () => {
    animate(
      "li", // target all <li> inside scope
      { opacity: 1, x: 0 },
```

```
      { delay: stagger(0.2), duration: 0.5 }
    );
  };

  return (
    <div>
      <ul ref={scope}>
        <li style={{ opacity: 0, transform: "translateX(-20px)" }}>Item 1</li>
        <li style={{ opacity: 0, transform: "translateX(-20px)" }}>Item 2</li>
        <li style={{ opacity: 0, transform: "translateX(-20px)" }}>Item 3</li>
      </ul>
      <button onClick={handleClick}>Animate</button>
    </div>
  );
}
```

In the above program-- About `stagger`--

> `stagger(delay, options)` is a helper function from Framer Motion used to **programmatically apply staggered delays** to multiple elements during animation (especially with `useAnimate()` or `animate()` calls).
>
> Syntax------ `stagger(delay, { startDelay, from, ease })`

Option	Description
`startDelay`	Delay before the first element starts
`from`	Where to start the stagger from:`"first"`,`"last"`,`"center"`, or an index
`ease`	Optional easing function

There is a timeline feature with which e coul orchestrate the animations--

--In **Framer Motion's** `useAnimate()` , the **timeline** is a way to define **sequential or overlapping animations** using an **array of steps** . It's similar to animation libraries like GSAP, where you can choreograph animations with precision — controlling when each starts, how long it lasts, and whether animations overlap.

🧠 Why Use Timeline?

Because `animate()` supports timelines, you can:

- Animate multiple elements in order or in parallel.
- Control the **exact timing** using the `at` key (like `+0.2`, `-0.1`, or a specific time).
- Avoid messy nested promises or `setTimeout`s.

SYNTAX---

```
animate([
  [target, keyframes, options],
  [target, keyframes, options],
  ...
])
```

Here each array element represents a step:

- `target` → selector (string), DOM node, or `scope.current`
- `keyframes` → animation values (`{ opacity: 1, x: 100 }`)
- `options` → transition settings (`{ duration: 0.5, at: "+0.2" }`)

EXAMPLE----

```
import { useAnimate } from "framer-motion";

export default function TimelineExample() {
  const [scope, animate] = useAnimate();

  const handleClick = () => {
    animate([
      ["#box1", { x: 100 }, { duration: 0.5 }],
      ["#box2", { y: 50 }, { duration: 0.5, at: "+0.2" }],
      ["#box3", { scale: 1.5 }, { duration: 0.5, at: "<" }],
    ]);
  };

  return (
    <div ref={scope}>
      <div id="box1" style={{ width: 50, height: 50, background: "red",
marginBottom: 10 }} />
      <div id="box2" style={{ width: 50, height: 50, background: "green",
marginBottom: 10 }} />
      <div id="box3" style={{ width: 50, height: 50, background: "blue" }} />
      <button onClick={handleClick}>Start Timeline</button>
    </div>
  );
}
```

⏱ **What does `at` mean?**

`at`value	Meaning
`"+0.2"`	Start 0.2s**after**the previous animation finishes

`at`value	Meaning
`"<"`	Start**at the same time**as the previous animation
`0.5`	Start**at an absolute time**(0.5 seconds from the timeline start)

10. Other Hooks

useVelocity()

The `useVelocity` hook in **Framer Motion** is used to **track the velocity (speed and direction)** of a `MotionValue`

It lets you:

- React to how **fast** a value is changing — for example, the speed of a drag.
- Create effects based on speed — like momentum, fling animations, or dynamic UI reactions.

useTime()

The `useTime` hook in **Framer Motion** provides a continuously updating timestamp — essentially a **live timer** that tells you how much time (in milliseconds) has passed since the component mounted.

🧠 **What it does:**

It gives you a **timestamp** that keeps increasing in real-time. Useful for creating effects that change over time, such as:

- Animating without `animate`
- Creating custom oscillations (like sine waves)
- Driving animations based on time elapsed

useAnimationFrame()

The `useAnimationFrame` hook lets you run **custom logic on every animation frame** — it's like `requestAnimationFrame`, but **React-safe** and integrated with Framer Motion's render cycle.

🧠 **What it's for:**

Use it when you want to:

- Run frame-by-frame logic (e.g., time-based movement).
- Animate things manually without using `motion.div` props.
- Create custom visual effects based on elapsed time.

useDragControls()

The `useDragControls` hook gives you **manual control over when a drag gesture starts** — instead of dragging the element immediately when the user touches it, you can **trigger drag manually** , like from a button or a specific region.

🔍 **Why use `useDragControls`?**

Use Case	Why it Helps
Drag should only start from a handle	You can trigger drag from any element
Prevent accidental drag	Only start drag on explicit user action
Fine-grained drag UX	Lets you control when & how drag begins
